Name: Chaitra Samant
Branch: TY Computer Engg
Reg No: 231070055
Batch: B

EXPERIMENT 09/10 - MIS LAB

AIM: To perform requirement analysis modeling and software design using structured or object-oriented approach for the Customer Complaint Intelligence System (CCIS) for Corporate Decision Making.

ANALYSIS MODELLING
Customer Complaint Intelligence System for Corporate Decision Making

THEORY

i) Requirement Analysis

Requirement Analysis is the process of determining user expectations for a new or modified product. It involves identifying, documenting, and managing the needs and requirements of all stakeholders to ensure the software system meets its intended goals. In the context of the CCIS, requirement analysis captures what the system must do (functional) and how well it must perform (non-functional) by engaging stakeholders such as customer service representatives, department managers, and the IT team.

ii) Structured Analysis

Structured Analysis is a classical approach to systems analysis that uses graphical tools, primarily Data Flow Diagrams (DFDs) and Entity-Relationship (ER) diagrams ,to model system behavior. It focuses on how data moves through a system and how it is transformed at each processing step. The approach decomposes a complex system into manageable modules represented as processes in a DFD. For CCIS, structured analysis helps model how complaints flow from submission through classification, analysis, and reporting.

iii) Object-Oriented Analysis (OOA)

Object-Oriented Analysis models a system as a collection of interacting objects that represent real-world entities. Each object encapsulates both data (attributes) and behavior (methods). OOA uses UML diagrams such as Class Diagrams, Sequence Diagrams, and Use Case Diagrams to represent the system structure and interactions. For CCIS, OOA identifies key entities such as Complaint, Customer, Agent, Department, and Dashboard as classes with well-defined responsibilities and relationships.

iv) Software Design

Software Design is the process of defining the architecture, components, interfaces, and other characteristics of a system to satisfy specified requirements. It bridges the gap between requirements analysis and implementation. Good software design ensures modularity, maintainability, scalability, and performance. Software design for CCIS involves designing the database schema, API interfaces, UI layout, AI/ML processing pipeline, and reporting subsystem.

v) Structured Design

Structured Design is a design methodology that focuses on decomposing a system into hierarchical modules, each with a single, well-defined function. It follows principles such as high cohesion (each module performs one task well) and low coupling (modules are minimally dependent on each other). Tools like Structure Charts are used. For CCIS, structured design breaks the system into modules: Complaint Intake, Classification Engine, Analytics Module, Notification Service, and Management Dashboard.

vi) Object-Oriented Design (OOD)

Object-Oriented Design translates the OOA model into an implementable design. It refines class diagrams, assigns responsibilities using patterns like MVC (Model-View-Controller), defines relationships (inheritance, aggregation, composition), and defines method signatures. OOD for CCIS involves designing class hierarchies for complaint types, defining the service layer for AI classification, designing the observer pattern for real-time notifications, and structuring the data access layer for database operations.

TASK

i) Functional and Non-Functional Requirements

Functional Requirements (FR)

#	Requirement ID	Description
1	**FR-01** *Complaint Submission*	The system shall allow customers and agents to submit complaints via web form, email, or API with details such as category, description, priority, and attachments.
2	**FR-02** *Automated Classification*	The system shall automatically classify incoming complaints into predefined categories (Billing, Technical, Delivery, Service Quality) using an AI/NLP engine.
3	**FR-03** *Sentiment Analysis*	The system shall analyze the sentiment of complaint text (Positive, Neutral, Negative, Critical) and attach a sentiment score to each complaint record.
4	**FR-04** *Complaint Assignment*	The system shall automatically or manually assign complaints to the relevant department or agent based on category, workload, and priority rules.
5	**FR-05** *Priority Escalation*	The system shall escalate high-priority or unresolved complaints automatically after a configurable SLA breach period, notifying supervisors via email/SMS.
6	**FR-06** *Root Cause Identification*	The system shall detect and tag recurring complaint themes and cluster similar complaints to identify underlying root causes for management review.
7	**FR-07** *Customer Churn Prediction*	The system shall use predictive analytics to calculate a churn risk score for each customer based on complaint frequency, severity, and resolution history.
8	**FR-08** *Executive MIS Dashboard*	The system shall provide role-based dashboards displaying KPIs such as total complaints, resolution rate, average resolution time, and department performance.
9	**FR-09** *Report Generation*	The system shall generate automated PDF/Excel reports (daily, weekly, monthly) summarizing complaint trends, top issues, and team performance metrics.
10	**FR-10** *Resolution Tracking*	The system shall track the status of each complaint through its lifecycle (Open, In-Progress, Escalated, Resolved, Closed) and maintain a full audit trail.
11	**FR-11** *Feedback Collection*	The system shall send post-resolution satisfaction surveys to customers and record their feedback scores and comments for quality tracking.

12	**FR-12** *User Authentication & RBAC*	The system shall enforce role-based access control with distinct roles: Customer, Agent, Supervisor, Department Manager, and System Admin.

Non-Functional Requirements (NFR)

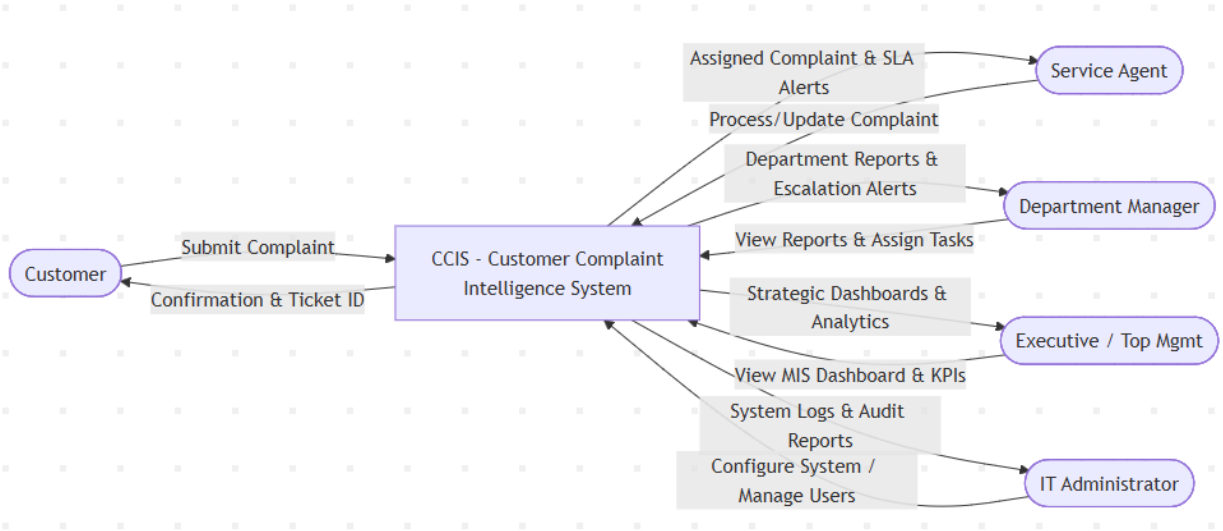
#	Category	Requirement
1	**Performance**	The system shall handle up to 10,000 concurrent users and process complaint submissions within 2 seconds under normal load conditions.
2	**Security**	All data must be encrypted at rest (AES-256) and in transit (TLS 1.3). Authentication must use JWT tokens with a 30-minute expiry and optional MFA.
3	**Availability**	The system shall maintain 99.9% uptime (less than 8.7 hours downtime per year) with automated failover and disaster recovery capabilities.
4	**Scalability**	The system architecture shall support horizontal scaling to accommodate a 300% increase in complaint volume without architectural changes.
5	**Usability**	The dashboard and complaint portal shall score at least 80 on the System Usability Scale (SUS) in user acceptance testing across all roles.

ii) Data Flow Diagrams (DFD)

Data Flow Diagrams model the flow of data through the CCIS. External entities, processes, and data stores are connected by data flows to represent how the system transforms inputs into outputs.

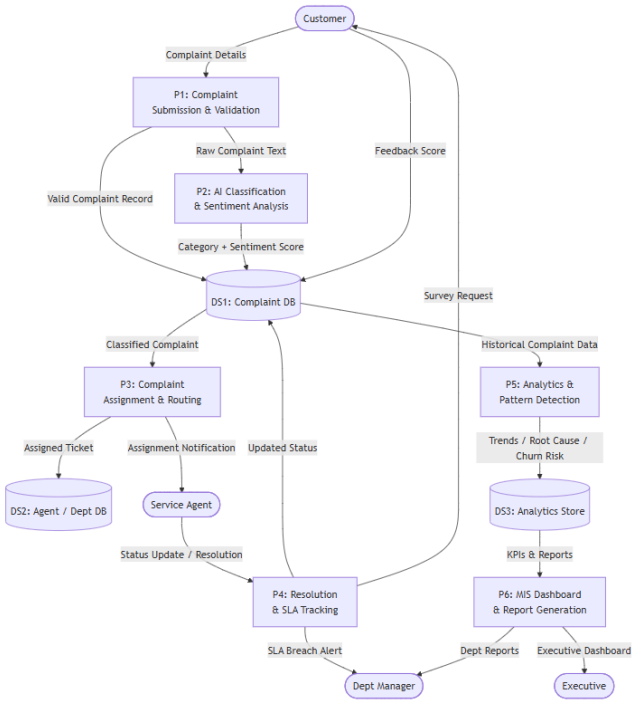
DFD Level 0 - Context Diagram

The Context Diagram shows the CCIS as a single process (black box) with external entities and the data flows between them.



DFD Level 1 - System Decomposition

DFD Level 1 decomposes the CCIS into its major processing modules, showing how data flows between them and the data stores.

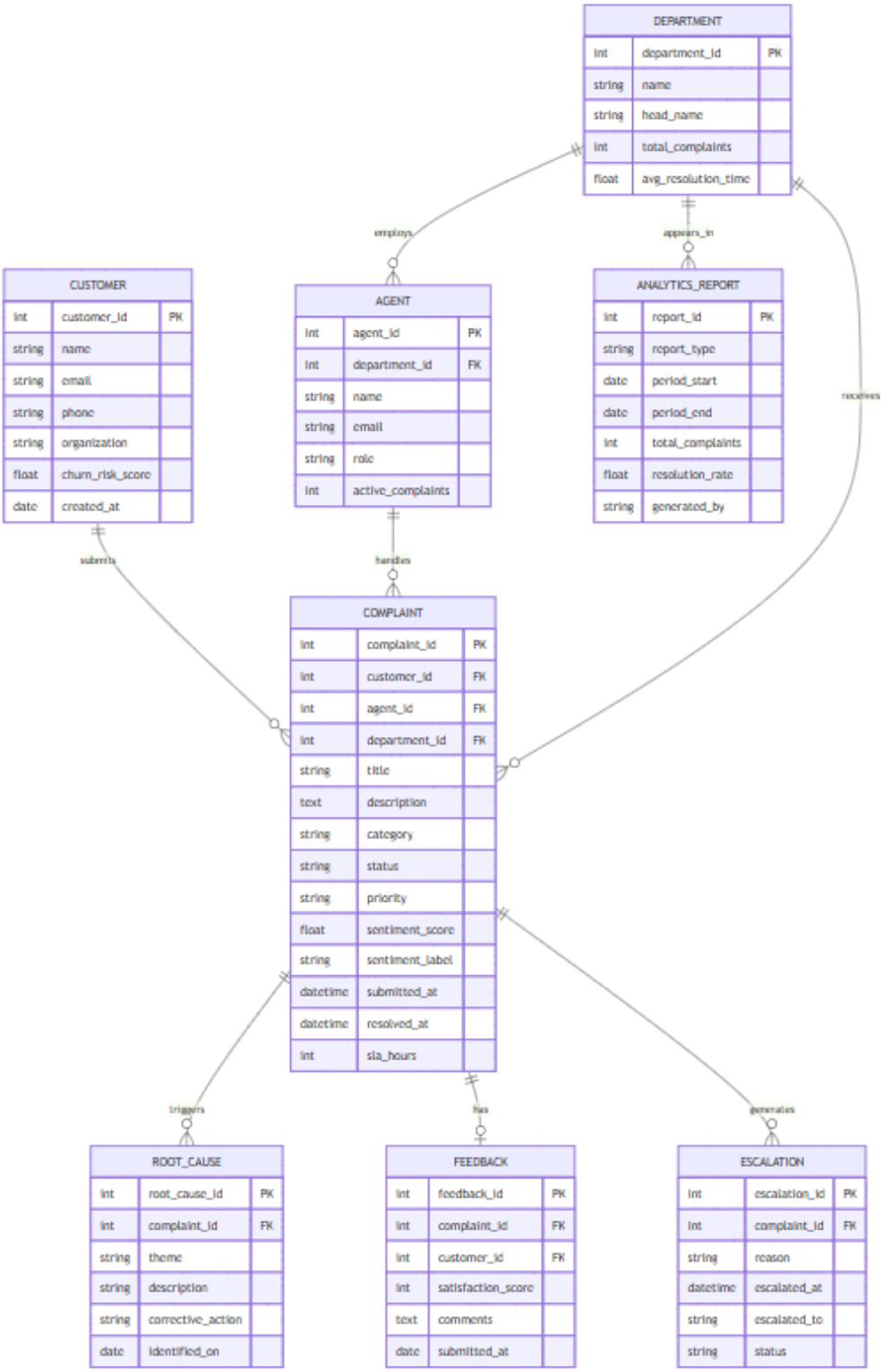


The six main processes identified in DFD Level 1 are:

- P1 - Complaint Submission & Validation: Receives raw complaints, validates mandatory fields, and stores them in the Complaint DB.
- P2 - AI Classification & Sentiment Analysis: Applies NLP models to classify complaint category and evaluate sentiment score.
- P3 - Complaint Assignment & Routing: Routes classified complaints to the appropriate department/agent based on category and workload.
- P4 - Resolution & SLA Tracking: Tracks complaint lifecycle, monitors SLA timers, and triggers escalations on breach.
- P5 - Analytics & Pattern Detection: Aggregates complaint data to detect recurring issues, root causes, and customer churn risk.
- P6 - MIS Dashboard & Report Generation: Synthesizes KPIs and produces dashboards and automated reports for management.

iii) Entity-Relationship (ER) Diagram

The ER Diagram defines the data entities in the CCIS, their attributes, and the relationships between them.


DEPARTMENT
int department_id PK
string name
string head_name
int total_complaints
float avg_resolution_time
employs
appears_in
receives
CUSTOMER
int customer_id PK
string name
string email
string phone
string organization
float churn_risk_score
date created_at
AGENT
int agent_id PK
int department_id FK
string name
string email
string role
int active_complaints
ANALYTICS_REPORT
int report_id PK
string report_type
date period_start
date period_end
int total_complaints
float resolution_rate
string generated_by
submits
handles
COMPLAINT
int complaint_id PK
int customer_id FK
int agent_id FK
int department_id FK
string title
text description
string category
string status
string priority
float sentiment_score
string sentiment_label
datetime submitted_at
datetime resolved_at
int sla_hours
triggers
has
generates
ROOT_CAUSE
int root_cause_id PK
int complaint_id FK
string theme
string description
string corrective_action
date identified_on
FEEDBACK
int feedback_id PK
int complaint_id FK
int customer_id FK
int satisfaction_score
text comments
date submitted_at
ESCALATION
int escalation_id PK
int complaint_id FK
string reason
datetime escalated_at
string escalated_to
string status

Key entities and their roles:

- CUSTOMER: Represents individuals or organizations raising complaints. Tracks churn risk score.
- COMPLAINT: Central entity containing full complaint lifecycle data including AI-generated sentiment and category.
- AGENT: Service personnel assigned to handle complaints within a department.
- DEPARTMENT: Organizational unit responsible for specific complaint categories.
- ROOT_CAUSE: Records the underlying issues identified from complaint clusters.
- FEEDBACK: Customer satisfaction score collected post-resolution.
- ESCALATION: Records each time a complaint is escalated beyond normal SLA.
- ANALYTICS_REPORT: Stores generated periodic reports for management review.

iv) Classes, Class Diagram & Sequence Diagram

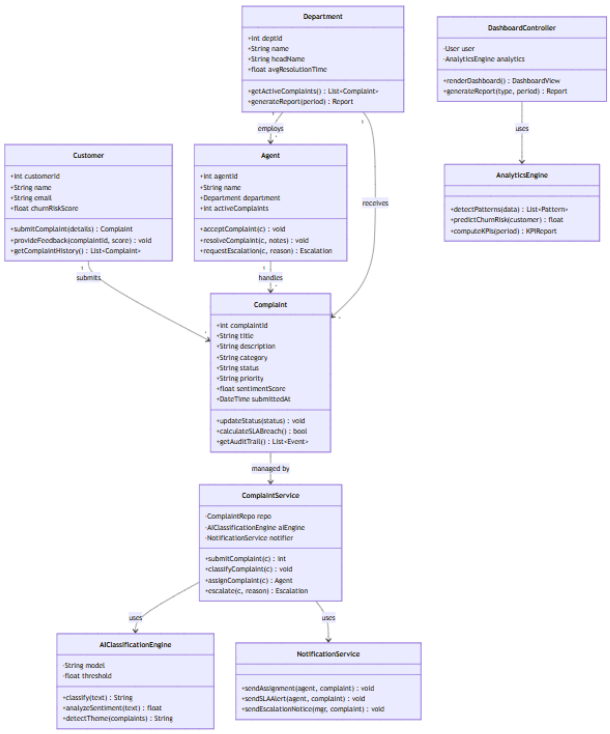
Identified Classes

The following key classes are identified in the CCIS using Object-Oriented Analysis:

Class Name	Key Attributes	Key Methods
ComplaintService	complaintRepo, aiEngine, notifier	submitComplaint(), classifyComplaint(), assignComplaint(), escalate()
Complaint	id, description, category, status, priority, sentimentScore	updateStatus(), calculateSLABreach(), getAuditTrail()
Customer	customerId, name, email, churnRiskScore	submitComplaint(), provideFeedback(), getComplaintHistory()
Agent	agentId, name, department, activeComplaints	acceptComplaint(), resolveComplaint(), requestEscalation()
Department	deptId, name, head, avgResolutionTime	getActiveComplaints(), generateReport(), updateSLA()
AIClassificationEngine	model, threshold, categories	classify(), analyzeSentiment(), detectTheme()
NotificationService	emailSender, smsSender	sendAssignment(), sendSLAAlert(), sendEscalationNotice()
AnalyticsEngine	dataStore, mlModel	detectPatterns(), predictChurnRisk(), computeKPIs()
DashboardController	user, analyticsEngine	renderDashboard(), generateReport(), filterByPeriod()
FeedbackService	feedbackRepo	sendSurvey(), recordFeedback(), computeSatisfactionScore()

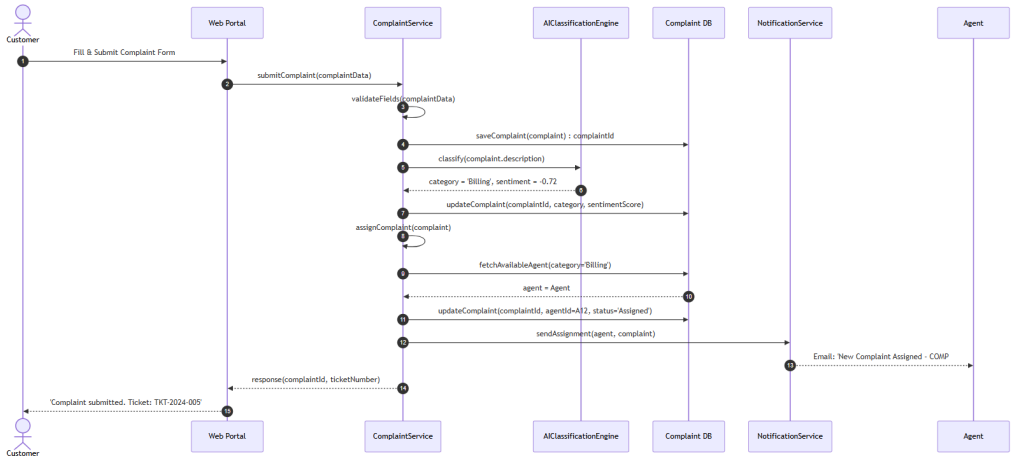
Class Diagram

The class diagram below shows the main classes of the CCIS system, their attributes, methods, and inter-class relationships including association, dependency, and composition.



Sequence Diagram - Complaint Submission Flow

The following sequence diagram illustrates the interaction between objects when a customer submits a complaint, the system classifies it, assigns it to an agent, and sends notifications



v) System Flow Explanation

The CCIS operates through a well-defined end-to-end flow that transforms raw customer complaints into strategic management insights. The complete flow is described in the following stages:

Stage 1: Complaint Intake

A customer submits a complaint through the web portal, mobile app, or via email integration. The system's Complaint Submission module validates the input (mandatory fields: category, description, contact details), assigns a unique ticket number, and persists the record to the Complaint Database. An acknowledgement with the ticket ID is immediately returned to the customer.

Stage 2: AI Classification and Sentiment Analysis

Once stored, the raw complaint text is passed to the AI Classification Engine. An NLP model (e.g., fine-tuned BERT or a rule-based classifier) assigns one of the predefined categories: Billing, Technical Support, Delivery, Service Quality, or Other. Simultaneously, a sentiment analysis model scores the text on a scale of -1 (very negative) to +1 (positive), and labels it as Critical, Negative, Neutral, or Positive. These results are written back to the complaint record.

Stage 3: Assignment and Routing

The Complaint Assignment module reads the category and routes the complaint to the corresponding department. Within that department, the system selects the agent with the lowest active complaint count (load balancing) and assigns the ticket. The assignment triggers an automated notification (email and/or in-app alert) to the selected agent, including the complaint details and SLA deadline.

Stage 4: Resolution and SLA Monitoring

The assigned agent reviews the complaint, communicates with the customer if needed, and works toward resolution. The SLA Tracker continuously monitors time elapsed since assignment. If the SLA threshold is breached, an escalation is automatically created, and the department supervisor is notified. The agent updates the complaint status through its lifecycle: In-Progress, Escalated, Resolved, or Closed. All status changes are recorded in the audit trail.

Stage 5: Feedback Collection

Upon complaint closure, the Feedback Service dispatches a post-resolution satisfaction survey to the customer via email. The customer rates their experience (1-5 stars) and optionally provides comments. Feedback is stored and linked to the complaint record, contributing to the agent and department performance metrics.

Stage 6: Analytics and Pattern Detection

The Analytics Engine periodically processes aggregated complaint data to identify recurring issues and root causes. It clusters similar complaints using semantic similarity, identifies themes (e.g., 'billing portal errors', 'delayed shipments'), and flags them for Root Cause Analysis. The engine also runs a churn risk model that scores each customer based on their complaint frequency, unresolved issues, and negative sentiment history. High-risk customers are flagged for proactive intervention by the retention team.

Stage 7: MIS Dashboard and Reporting

The Dashboard Controller aggregates KPIs from the Analytics Store and renders role-based views. Executives see strategic summaries: total complaints by category, resolution rate trends, churn risk heat maps, and financial impact estimates. Department managers see their team's performance metrics. Agents see their personal queue and pending SLAs. The system also auto-generates daily, weekly, and monthly PDF/Excel reports that are emailed to relevant stakeholders on schedule.

In summary, the CCIS flow ensures that every customer complaint is captured, intelligently processed, efficiently resolved, and comprehensively analyzed. The feedback loop from analytics back into operations enables continuous improvement of service quality and customer satisfaction, while MIS dashboards empower data-driven corporate decisions at all organizational levels.

CONCLUSION

This experiment focused on requirement analysis and system design for the Customer Complaint Intelligence System (CCIS). A complete set of functional and non functional requirements was identified, covering complaint handling, AI classification, SLA tracking, analytics, and reporting. Data Flow Diagrams defined system boundaries and core processes, while the ER diagram established the database structure. Object oriented analysis further detailed system components through class and sequence diagrams.

Overall, the exercise highlighted the importance of structured design before implementation. The combination of DFDs and object oriented models provides a clear blueprint for development. For CCIS, this structured approach ensures that complex features like AI driven insights and decision support can be built efficiently and reliably.

.